

Gradient Boosting

Author: Rohit Mande
Founder and CEO, intrvu.ai

Website: <https://intrvu.ai>



Index

- What is boosting? ✓
- AdaBoost ✓
- Gradient Boosting ✓
- XGBoost ✓
- Features importance ✓

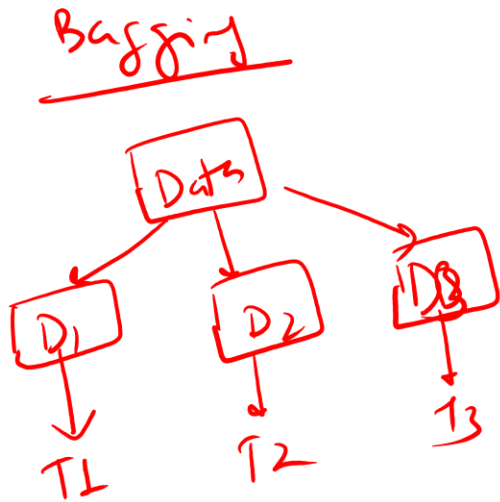
INTTRVU.ai

Website: <https://inttrvu.ai>

→ Ensemble Learning Method

Ensemble of trees → All trees are very weak learners/models/predictors

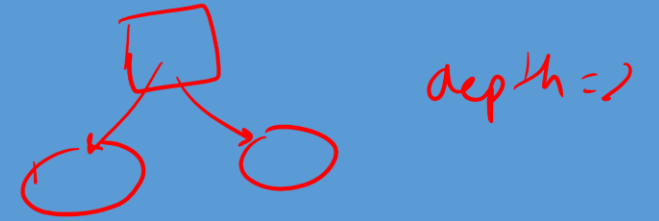
Combination of these weak learners make up a strong model.



What is boosting?

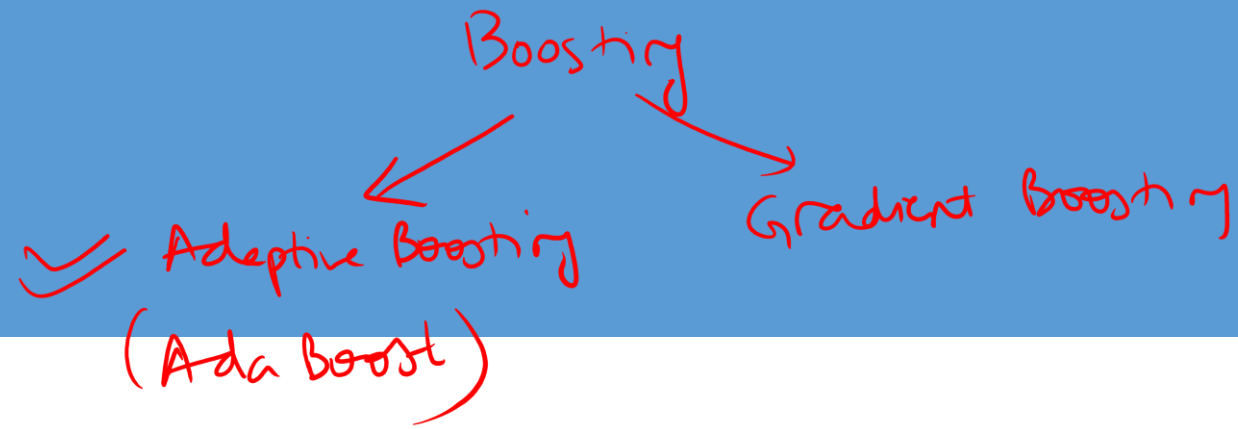
Website: <https://inttrvu.ai>

What is boosting?



- Boosting is ensemble learning method where multiple weak learners are trained sequentially to get the prediction
- Boosting is supervised learning method
- Boosting can be used to build regression as well as classification models
- Multiple weak learners create a strong learner (model with low bias and low variance)

Example of weak learner: decision tree with small depth

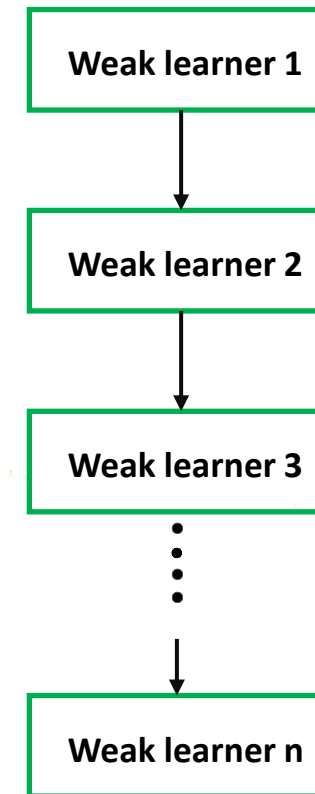


Adaboost

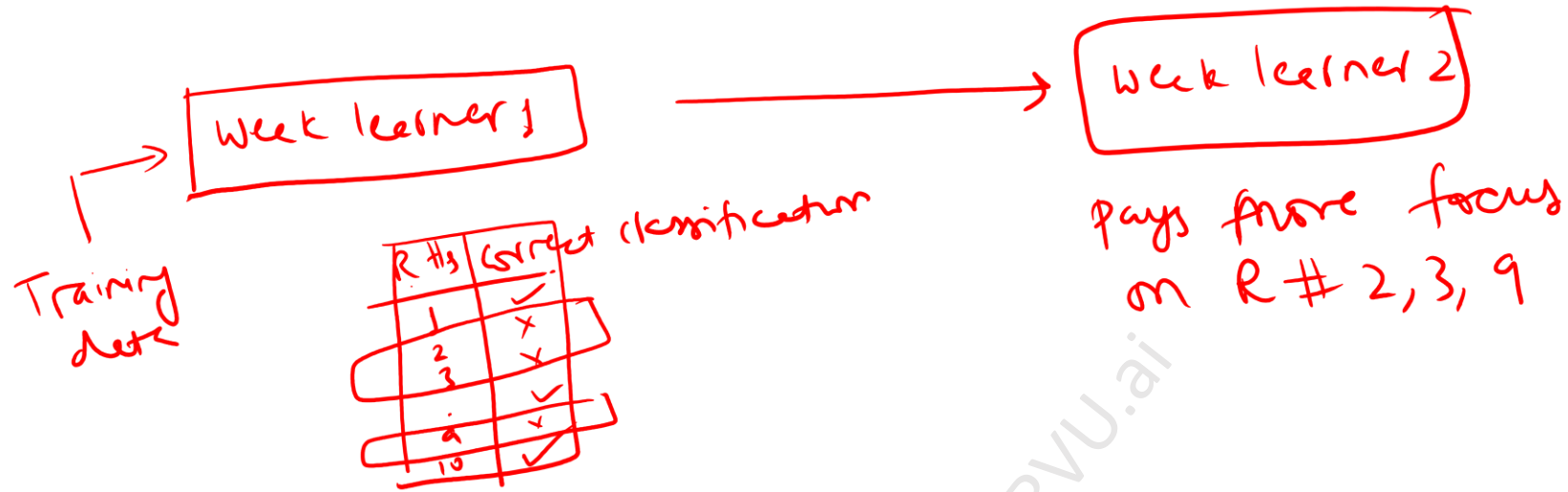
Website: <https://inttrvu.ai>

AdaBoost

- AdaBoost stands for Adaptive Boosting
- AdaBoost is mainly used for binary classification (can be used for regression as well)
- AdaBoost is Adaptive as while training subsequent weak learners more weightage (importance) is given to misclassified training samples from previous weak learner



Adaptive $\Rightarrow$ weights



How can a model change its focus on different examples?
The focus can be changed by changing the weights

Linear Regression

$$L = \frac{1}{N} \sum (y_i - \hat{y}_i)^2 \Rightarrow L = \sum \underset{\uparrow}{w_i} (y_i - \hat{y}_i)^2$$

change these weights to change the focus

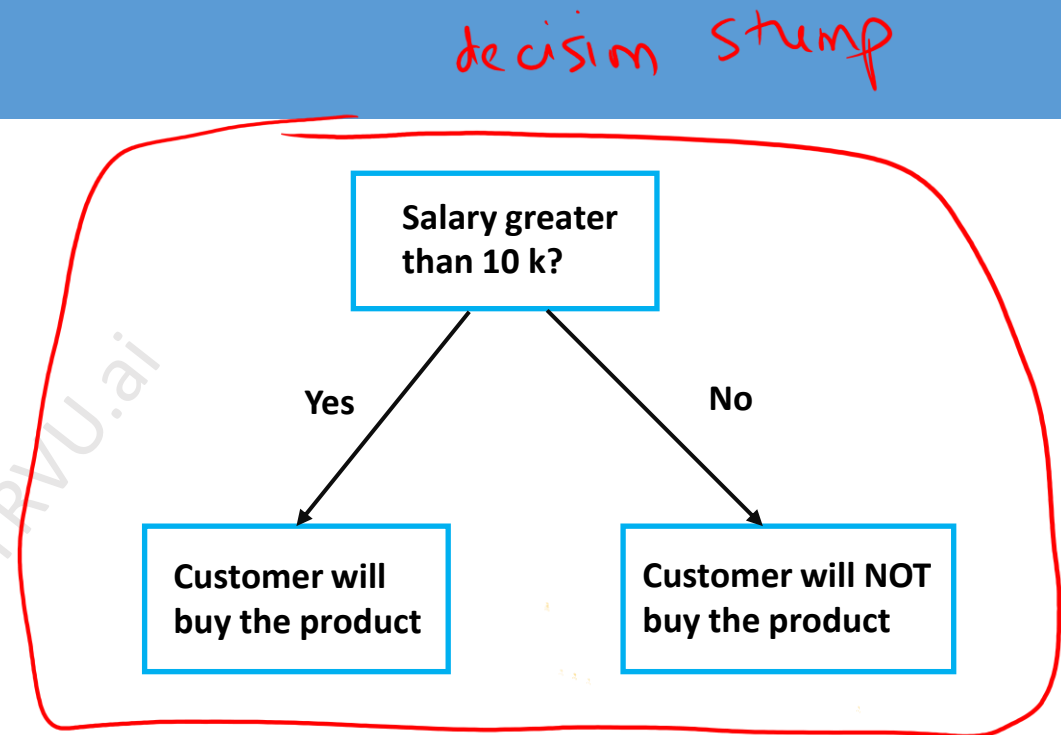
In AdaBoost, we are going to change / increase the weights of the misclassified samples

INTTRVU.ai

Website: <https://inttrvu.ai>

AdaBoost

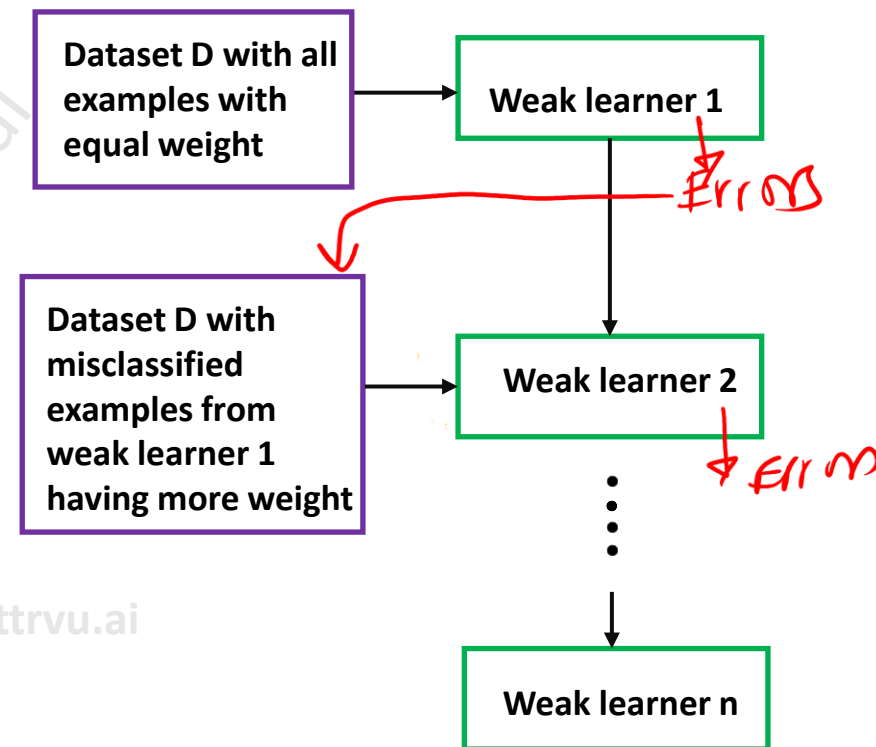
- AdaBoost typically uses decision stumps as a weak learner
- Decision stumps are one-leave decision trees



Website: <https://inttrvu.ai>

AdaBoost

- AdaBoost increases the weight of misclassified examples from previous learner (focus on correcting the mistakes from previous weak learner)



Website: <https://inttrvu.ai>

Adaptive Boosting Algorithm

① Step 1 $\Rightarrow w_i = \frac{1}{M} \quad \forall i$

$i = 1$ to M $\nwarrow$ total number of training samples

② Step 2 $\Rightarrow$ Train decision stump 1

③ Step 3 calculate errors

$$\text{Error} = \frac{\sum_{i=1}^M w_i I(y_i \neq h(x_i))}{\sum_{i=1}^M w_i}$$

$h(x_i)$ = prediction for x_i made by decision stump 1

x_i = i th row

y_i = i th label

If $y_i \neq h(x_i) \Rightarrow I = 1$
also $I = 0$

In essence, we are adding the weights of misclassified samples

Step 4

$$\alpha = \frac{1}{2} \ln \left(\frac{1 - \text{Error}}{\text{Error}} \right)$$

$$\alpha = \frac{1}{2} \ln \left(\frac{1}{\text{Error}} - 1 \right)$$

α = contribution of the decision stump towards the final prediction

$\ln \rightarrow$ natural log

If Error = 0.5

$$\boxed{\alpha = 0}$$

If Error < 0.5

$$\Rightarrow \alpha = \frac{1}{2} \ln \left(\frac{1 - \text{Error}}{\text{Error}} \right) > 0$$

If Error > 0.5

$\Rightarrow \alpha < 0 \Rightarrow \nexists$ (revisit later)
option 1 $\rightarrow$ discard this tree

option 2 $\Rightarrow$ invert the predictions

Step 5

$$w_i^{\text{new}} = w_i^{\text{old}} \exp(\alpha I(y_i \neq h(x_i)))$$

Case I $y_i = h(x_i) \Rightarrow \text{prediction} = \text{actual label}$

$$I(y_i \neq h(x_i)) = 0$$

$$\Rightarrow w_i^{\text{new}} = w_i^{\text{old}} \underbrace{\exp(\alpha \cdot 0)}_{= 1} = w_i^{\text{old}}$$

Case II $y_i \neq h(x_i) \Rightarrow I(y_i \neq h(x_i)) = 1$

$$\Rightarrow w_i^{\text{new}} = w_i^{\text{old}} \exp(\alpha \cdot 1) = w_i^{\text{old}} e^{\alpha}$$

$$\Rightarrow w_i^{\text{new}} = w_i^{\text{old}} \cdot e^{\alpha}$$

Website: <https://inttrvu.ai>

Normalization step

$$w_i^{\text{new}'} = \frac{w_i^{\text{new}}}{\sum w_i^{\text{new}}}$$

How to make final prediction?

$$H(x_i) = \text{sign} \left(\sum_{j=1}^T \alpha_j h_j(x_i) \right)$$

$\uparrow$
jth row

$\uparrow$
weight of a stump

$\uparrow$
total number of decision stumps you made

$\uparrow$
prediction of jth decision stump for x_i

Website: <https://inttrvu.ai>

Q What would you do if your classification model is 95% inaccurate?

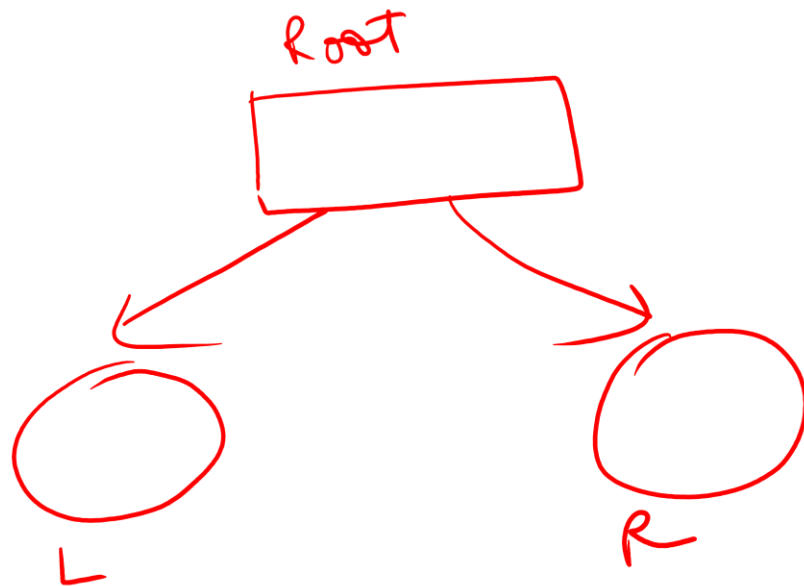
Actual	Prediction	
	0	1
1	0	1
1	0	1
1	0	1
1	0	1
1	0	1
1	0	1

INTTRVU.ai

Website: <https://inttrvu.ai>

How to train an individual decision stump?

$$\text{Gini impurity} = 1 - \sum_{i=1}^n p_i^2$$



$$\Delta G = G_{\text{root}} - \left[\frac{N_L}{N_L + N_R} G_L + \frac{N_R}{N_L + N_R} G_R \right]$$

for decision stump

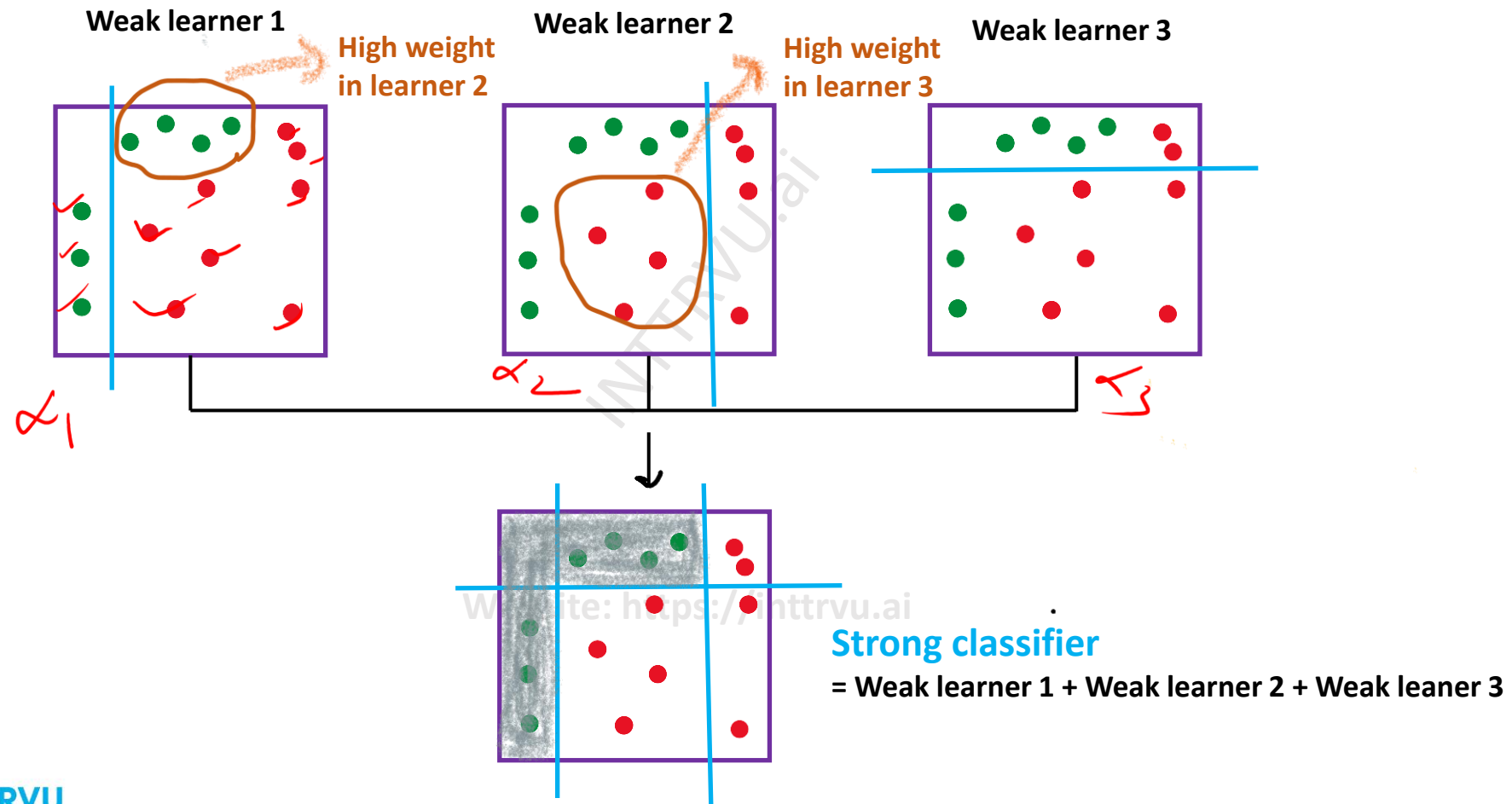
$$\Delta G = G_{\text{root}} - \left[\frac{\sum_{i \in L} w_i}{\sum_{i \in L} w_i + \sum_{j \in R} w_j} G_L + \frac{\sum_{j \in R} w_j}{\sum_{i \in L} w_i + \sum_{j \in R} w_j} G_R \right]$$

The diagram shows the same formula as above, but with the denominators of the fractions circled. Below each circled denominator, there is a double vertical line and the number 1, indicating that the sum of weights in each partition equals 1.

Website: <https://inttrvu.ai>

$$\Rightarrow \Delta G = G_{\text{root}} - \left[\left(\sum_{i \in L} w_i \right) G_L + \left(\sum_{j \in R} w_j \right) G_R \right]$$

AdaBoost



Gradient Boosting

Website: <https://inttrvu.ai>

What is Gradient boosting?

- Gradient Boosting is ensemble learning method where multiple weak learners are trained sequentially to get the prediction
- It can be used for building regression as well as classification models
- When decision tree is used as weak learner it is called as gradient boosting trees
- Gradient boosting uses gradient descent for the optimization

Gradient boosting regression

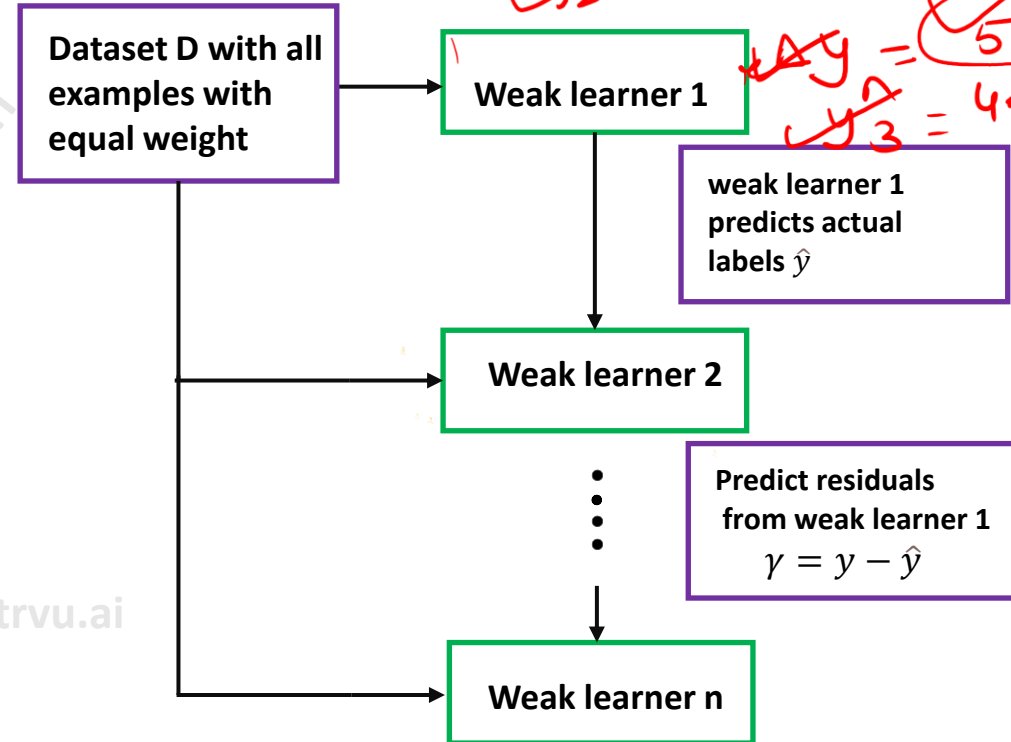
$$y = \hat{y}_1 + \hat{y}_2 + \hat{y}_3 = 95000$$

No discussion for weights

$$y = 100,000 \checkmark$$
$$\hat{y}_1 = 80,000 \checkmark$$

$$\Delta y = y - \hat{y}_1 = 20,000 \checkmark$$
$$\hat{y}_2 = 15,000 \checkmark$$
$$\Delta y = 5,000 \checkmark$$
$$\hat{y}_3 = 4,500 \checkmark$$

- Each weak learner tries to predict residual value (error) from previous learner (focus on correcting the mistakes from previous weak learner)
- Weak learner 1 gets actual features as input and it is trained to predicts actual labels
- Weak learner 2 onwards gets actual features as input and try to predict residual value (error) from previous weak learner



Website: <https://inttrvu.ai>

Note: The residuals for classification in gradient boosting are the differences between the predicted class probabilities and the true class labels.

Gradient boosting regression

$$F_{m+1}(x_i) = F_m(x_i) + h_m(x_i)$$

m is stage of weak learner ($m = 2$ will give output by adding predictions from all weak learners till 3)

x_i is input features

$h_m(x_i)$ is current weak learner which is added to all previous learners

Weak learner 1

Weak learner 2

Weak learner n

Final prediction is addition of prediction from all weak learners

$$F_{m+1}(x_i) = \sum_{j=1}^m h_j(x_i)$$

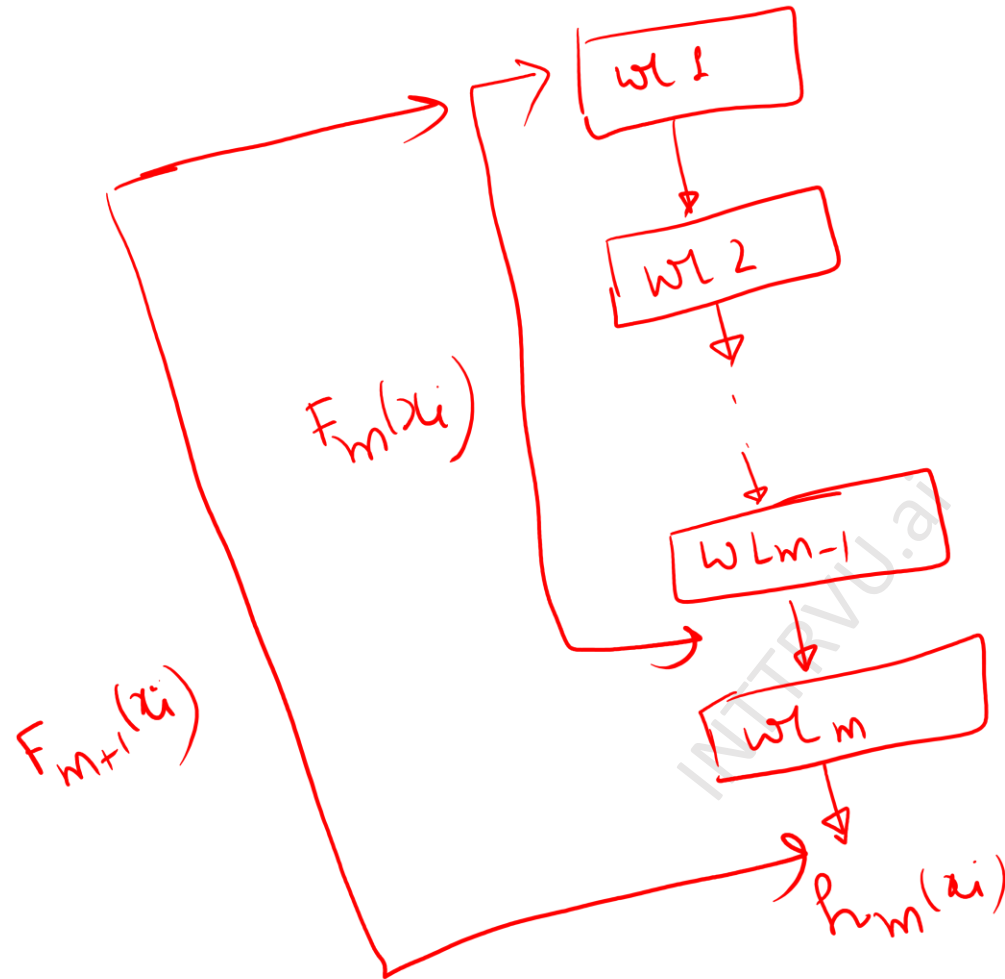
prediction

$F_m(x_i)$

prediction till $(m-1)^{th}$ weak learner

$x_i \Rightarrow i^{th}$ observation / i^{th} row

$h_m(x_i) \Rightarrow$ prediction made by m^{th} weak learner for x_i



Website: <https://inttrvu.ai>

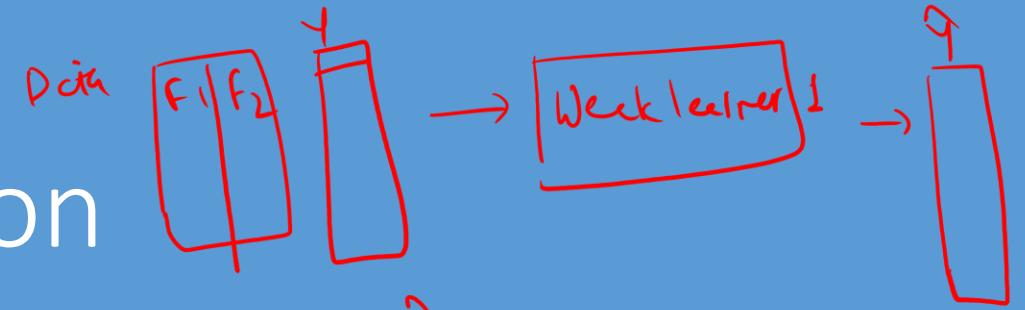
$$F_{m+1}(x_i) = F_m(x_i) + h_m(x_i)$$

$h_m(x_i)$ = prediction of m th week learner for x_i

$$F_m(x_i) = \sum_{j=1}^{m-1} h_j(x_i)$$

$$F_{m+1}(x_i) = \sum_{j=1}^m h_j(x_i)$$

Gradient boosting regression



final

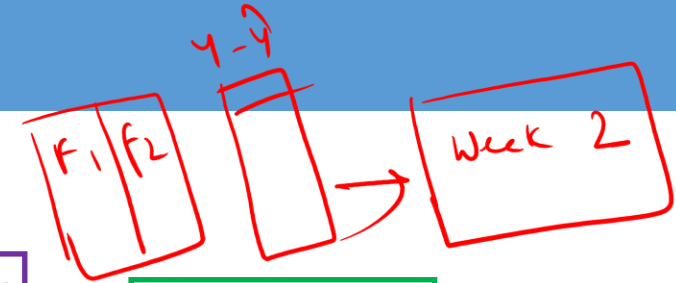
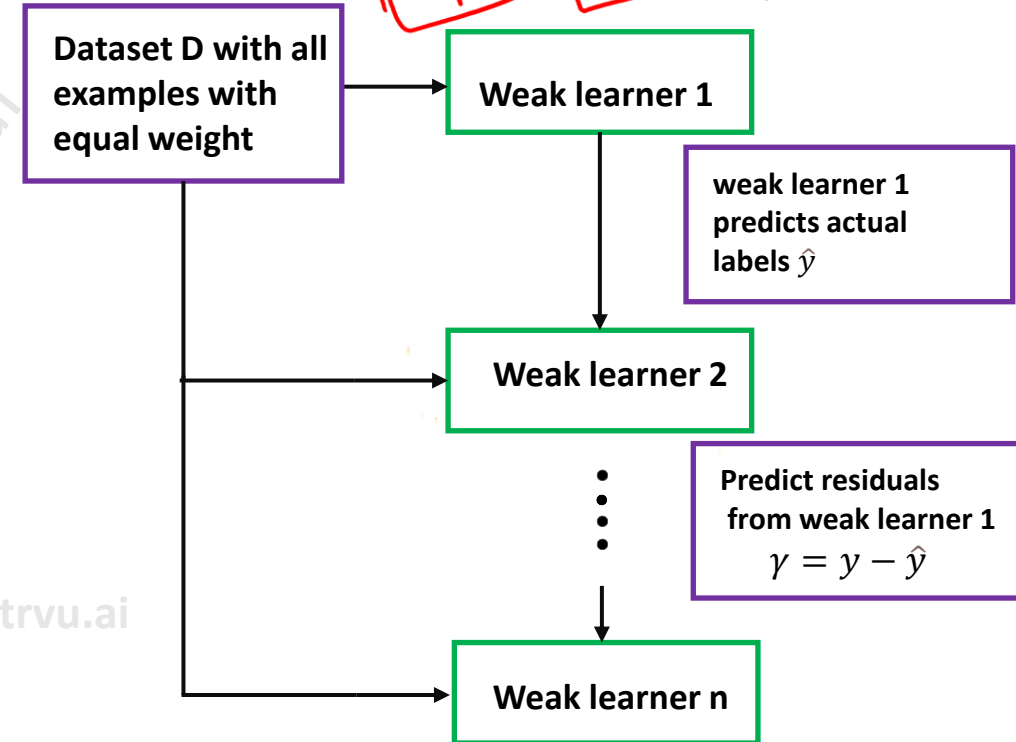
$$y_{pred} = \hat{y} + \hat{\gamma}_1 + \hat{\gamma}_2 + \dots$$

$\hat{y}$ is prediction from weak learner 1

$\hat{\gamma}_1$ is prediction from weak learner 2 which is trying to predict residual from weak learner 1

$\hat{\gamma}_2$ is prediction from weak learner 3 which is trying to predict residual from weak learner 1

Website: <https://inttrvu.ai>



Shrinkage/Learning rate

controlling the problem of overfitting

$v \downarrow \Rightarrow$ you will need to create more number of trees for final prediction

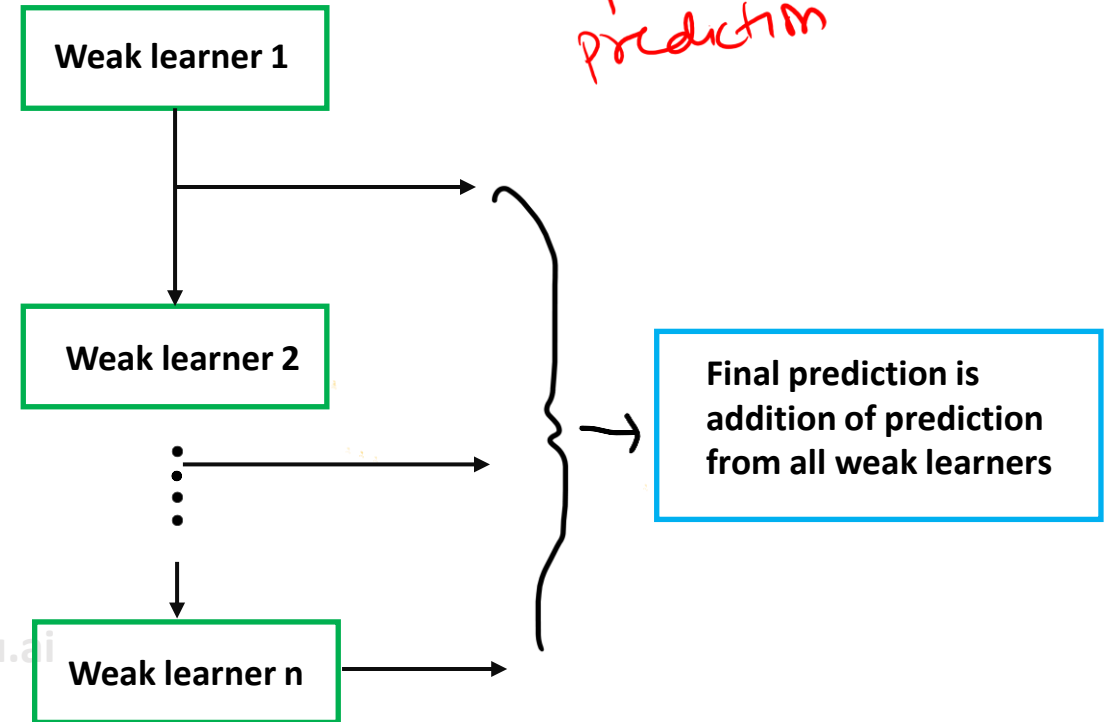
$$y_{pred} = \hat{y} + v\hat{y}_1 + v\hat{y}_2 + \dots$$

$$y_{pred} = \hat{y} + (1)\hat{y}_1 + (1)\hat{y}_2 + \dots$$

v is shrinkage parameter (also called as learning rate)

Prediction of each tree in the ensemble is shrunk after it is multiplied by the learning rate

Low learning rate might need large number of estimators (weak learners) to reach optimum output (as we will move towards minimum cost point slowly in gradient descent)



Algorithm for gradient boosting trees

↳ Decision trees as the weak learner

Step 1 Define the loss function (L)

$$L(y, \hat{F}(x))$$

feature row

actual value

predicted value

Regression

$$L(y, F(x)) = \frac{1}{2N} \sum_{i=1}^N (y_i - F(x_i))^2 \Rightarrow \text{loss for entire dataset}$$

Classification

$$L(y, F(x)) = -\frac{1}{N} \sum [y_i \log p_i + (1-y_i) \log(1-p_i)]$$

Website: <https://inttrvu.ai>

$$p_i = \frac{1}{1 + e^{-F(x)}}$$

$$L(y, F(x)) = -\frac{1}{N} \sum \left[y_i \log \frac{1}{1 + e^{-F(x)}} + (1-y_i) \log \left(1 - \frac{1}{1 + e^{-F(x)}} \right) \right]$$

Step 2 Model Initialization

For regression $F_0(x) = \frac{1}{N} \sum_{i=1}^N y_i$

For classification $F_0(x) = \arg \min_c \sum_{i=1}^N L(y_i, c)$

→ Loss function
for entire data

$F_0(x)$ will be equal to that value of c which will minimize
 $\sum_{i=1}^N L(y_i, c)$

$$F_0(x) = c = \ln \frac{\bar{y}}{1 - \bar{y}} \Rightarrow \bar{y} = \sum_{i=1}^N y_i$$

Website: <https://inttrvu.ai>

Cost Function for classification

Let's take only 1 row

$$L(y_i, p_i) = -y_i \log(p_i) - (1-y_i) \log(1-p_i) \Rightarrow \text{Cross entropy}$$

$$\text{Total Loss} = \frac{1}{N} \sum_{i=1}^N L(y_i, p_i) = -\frac{1}{N} \sum y_i \log p_i + (1-y_i) \log(1-p_i)$$

INTTRVU.ai

Website: <https://inttrvu.ai>

Step 3

Regression $L(y_i, F(x_i)) = \frac{1}{2} (y_i - F(x_i)) ^2$

To get the label for m^{th} weak learner

$$\frac{\partial L}{\partial F} = y_i - F_{m-1}(x_i)$$

gradient descent

$$\Rightarrow r_{im} = y_i - F_{m-1}(x_i)$$

residual
value

i^{th} row

m^{th} weak
learner

predictions
till $(m-1)$

r_{im} will be the new label for m^{th} weak learner
or new target

classification

$$r_{im} = y_i - p_i = y_i - \frac{1}{1 + e^{-f_{m-1}(x_i)}}$$

Step 4 Fit the weak learner

A new decision tree $h_m(x)$ will be created to predict the residual values

$h_m(x)$ = predictions made by the m^{th} weak learner for x feature row

Website: <https://inttrvu.ai>

Step 5 Repeat step 3 and step 4 till you get the desired level of accuracy or desired number of trees.

INTTRVU.ai

Website: <https://inttrvu.ai>

XGBoost

Website: <https://inttrvu.ai>

What is XGBoost?

$$\checkmark \frac{\partial L}{\partial F(x)} \rightarrow \text{gradient} + \text{Hessians}$$
$$\frac{\partial^2 L}{\partial F^2(x)} \rightarrow$$

XGBoost stands for **Extreme Gradient Boosting**

Weak Learners: Decision trees ✓

to fight overfitting

✓ **Regularization:** XGBoost uses advanced regularization (L1 & L2), which improves model generalization capabilities.

Speed: Its training is very fast and can be distributed across clusters.

Optimization: It computes second-order gradients, i.e., second partial derivatives of the loss function which provides more information about the direction of gradients and how to get to the minimum of our loss function

Regularization

Objective = Loss Function + Regularization

$$\text{objective} = \frac{1}{2N} \sum (y_i - f(x_i))^2 + \sum_{k=1}^T \Omega(f_k)$$

Total number of trees

$\Omega(f_k) \rightarrow$ regularization term for k^{th} tree

For 1 tree

$$\Omega(f) =$$

$$\gamma(P) +$$

total number
of leaves in tree

$$\frac{1}{2} \lambda \sum_{j=1}^P (w_j)^2$$

weight of j^{th} leaf

$$\alpha \sum_{j=1}^P |w_j|$$

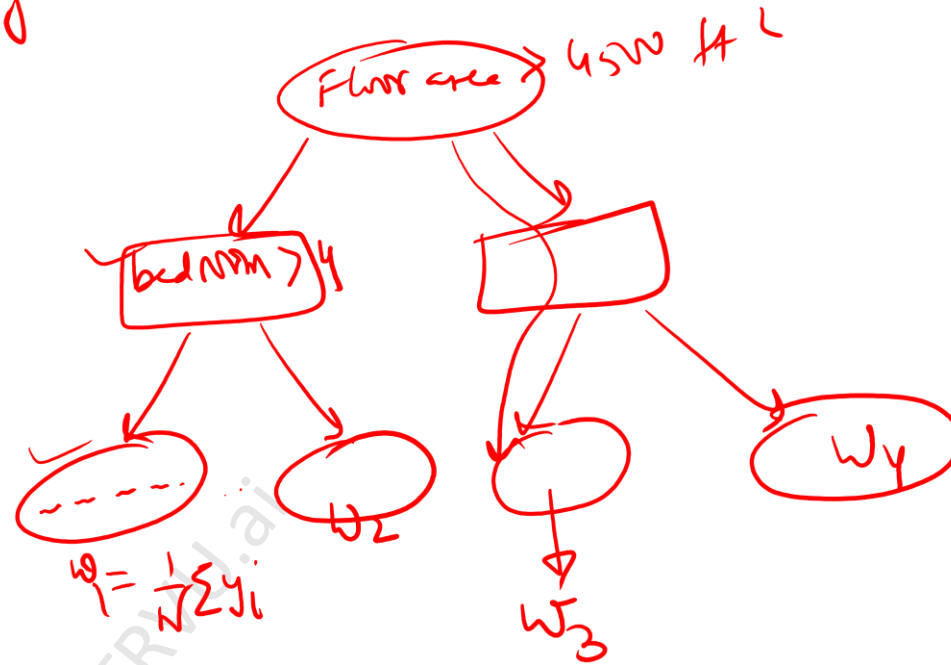
L1 term

$\gamma, \lambda, \alpha \Rightarrow$
regularization
coefficient

Website: <https://inttrvu.ai>

Significance/meaning of w_j

$w_j \Rightarrow$ What value you will provide to the new incoming datapoint if it ends up in j^{th} leaf



Website: <https://inttrvu.ai>

XGBoost uses Parallelism

It uses parallelism to train 1 week learner

Buging

WL1

WL2

WL3

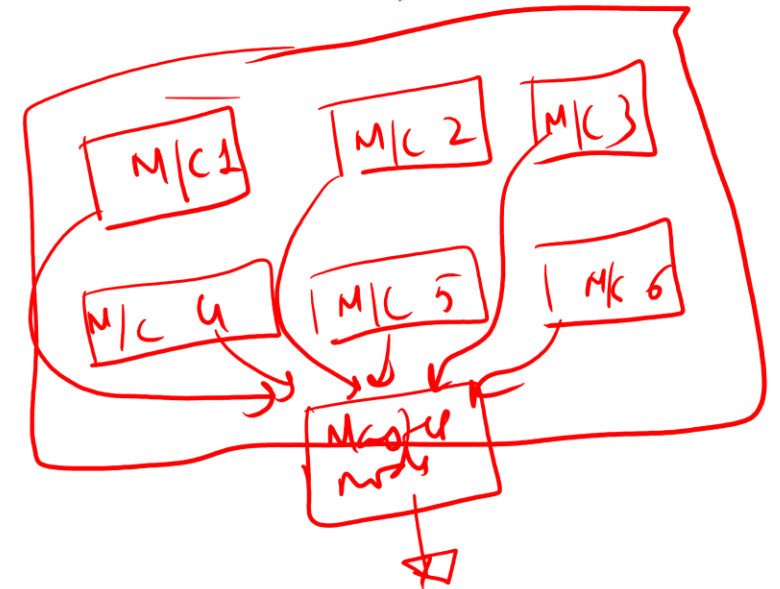
Boosting



- ① Data partitioning
- ② Feature partitioning.
- ③ Gradient and Hessian computation

Website: <https://inttrvu.ai>

cluster of machines



Hessians in XGBoost

$$\text{Most optimum weights} = - \frac{\sum g_i}{\sum h_i + \lambda}$$

$g_i \Rightarrow$ gradient of i^{th} row in the leaf j

$h_i \Rightarrow$ hessian of i^{th} row in the leaf j

$\lambda \Rightarrow$ L2 controlling parameter

$$L(y, f(x)) = \frac{1}{2} (y - f(x))^2$$

$$g = \frac{\partial L}{\partial f} = \frac{1}{2} \cdot 2 \cdot (y - f(x)) \cdot (-1) = -(y - f(x))$$

$$h = \frac{\partial^2 L}{\partial f^2} = 1$$

Website: <https://inttrvu.ai>

Advantages and Dis-advantages of XGBoost

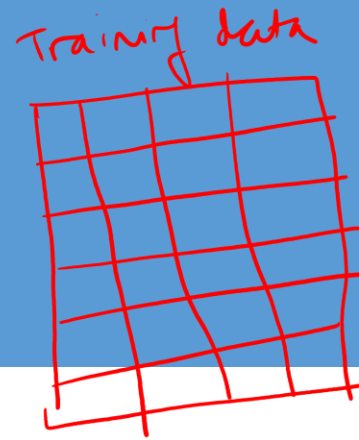
Advantages:

- Active production use in multiple organizations across various use cases
- Provides better results than random forest in many use cases
- Preferred classification/regression method when you have large dataset
- Can learn linear as well as non-linear decision rules/ relationships

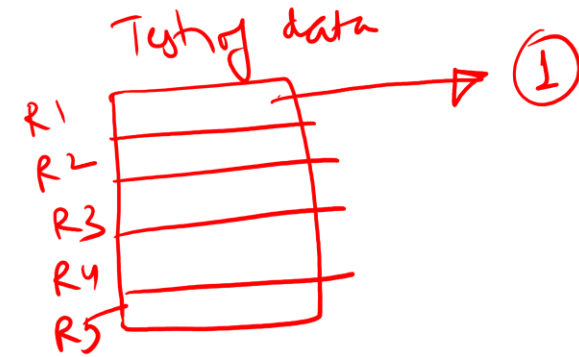
Dis-advantages:

- Highly complex model (large number of weak learners consisting of large depth trees) can lead to overfitting

Website: <https://inttrvu.ai>



Global feature importance



Feature Importance

Global

Local

Website: <https://inttrvu.ai>

At the entire
model level

Feature Importance

Three different methods:

①

Weight:

Relative number of times a particular feature is used for tree splitting in the model (least preferred)

②

Coverage:

Relative number of observations related to this feature. Deeper in the tree a node is, lower this metric will be

③

Gain:

Improvement in accuracy brought by a feature to the branches it is on (difference between accuracy before split and after split based on the feature)

Weight

We track the count of number of times a particular feature is used for splitting

50 weak learners

W_L 1 →

F ₁	1
F ₂	0
F ₃	2
F ₄	1

W_L 2 →

F ₁	2
F ₂	1
F ₃	0
F ₄	0

+

F ₁	3	3/7	
F ₂	1	1/7	~14
F ₃	2	2/7	~28
F ₄	1	1/7	~14
Total		7	

Website: <https://inttrvu.ai>

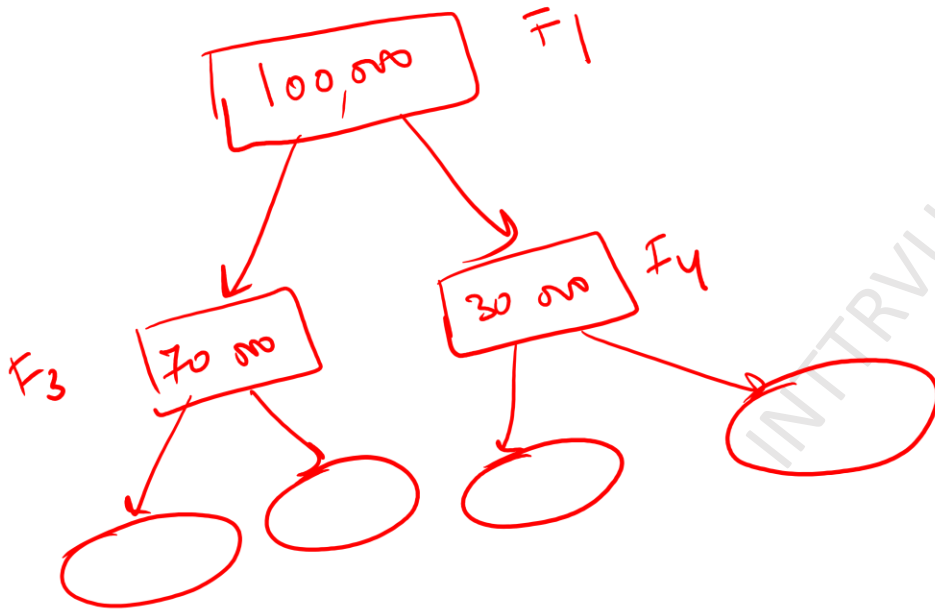


INTTRVU

Coverage

Instead of counting number of times ; We count number of data points
a particular splitter acted upon

50 WLS



		<u>wL1</u>
F_1	$\pm$	100,000
F_2	θ	0
F_3	$\pm$	70,000
F_4	$\pm$	30,000

		<u>wL2</u>
F_1		0
F_2		100,000
F_3		40,000
F_4		60,000

+

F_1	100,000
F_2	100,000
F_3	110,000
F_4	90,000

Website: <https://inttrvu.ai>

GAIN

Our concern is to measure impact in "some find metric"

Objective = Loss function + Regularization

A feature is more important to us if it provides more reduction in the objective function.

$$\text{objective} = \frac{1}{2N} \sum (y_i - \hat{F}(x_i))^2 + \frac{1}{T} \sum_{k=1}^T R(f_k)$$

$$\Delta G = \frac{1}{2} \left[\frac{G_L^2}{H_L + \lambda} + \frac{G_R^2}{H_R + \lambda} - \frac{(G_L + G_R)^2}{H_L + H_R + \lambda} \right]$$

WL1

F1	ΔG_{11}
F2	ΔG_{12}
F3	ΔG_{13}
F4	ΔG_{14}

+

<u>WL2</u>	
F1	ΔG_{21}
F2	ΔG_{22}
F3	ΔG_{23}
F4	ΔG_{24}

F1	$\Delta G_{11} + \Delta G_{21}$
F2	$\Delta G_{12} + \Delta G_{22}$
F3	$\Delta G_{13} + \Delta G_{23}$
F4	$\Delta G_{14} + \Delta G_{24}$

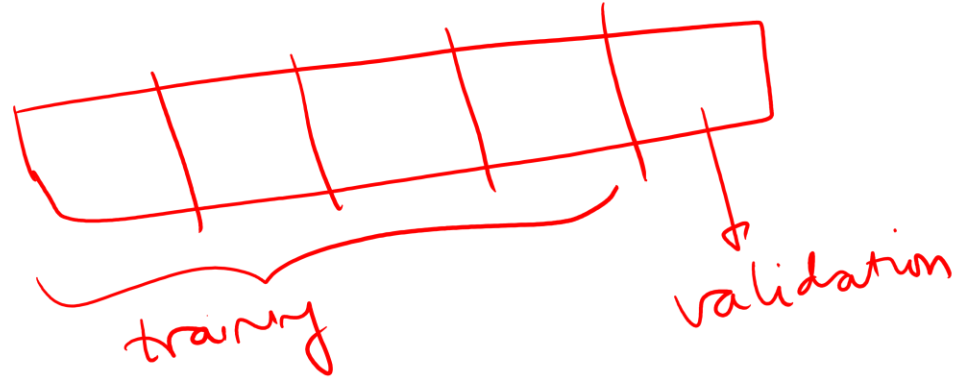
Questions?

Website: <https://inttrvu.ai>

refit parameter in Grid search CV

Grid search CV

$k=5$



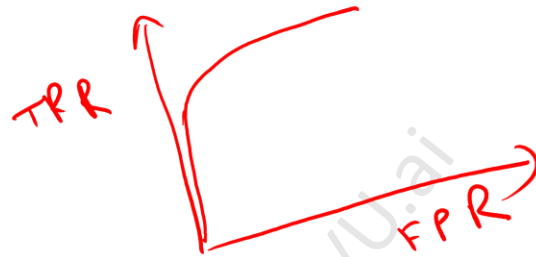
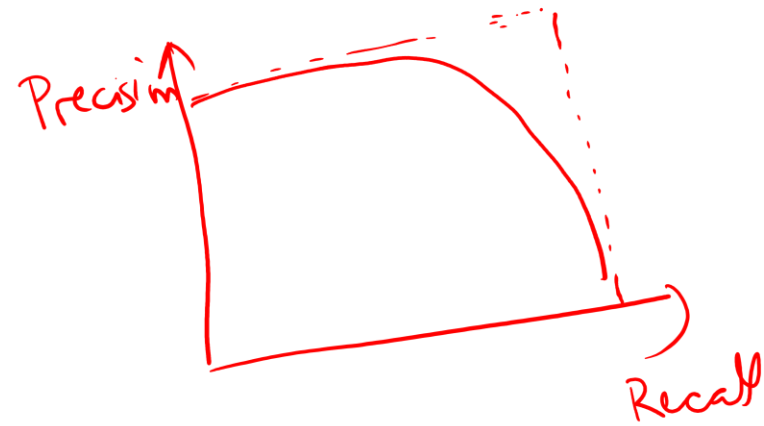
By comparing the validation scores, we get to know the best hyperparameter combination.

Set 10 → best hyperparameter
↳ model was trained only on 4 parts

If you want to use this model for test then retrain the model on all 5 parts

Precision recall curve

Data	$R_1 - R_{100}$ probabilities	Predictions
R_1	P_1	
R_2	P_2	
$\vdots$	$\vdots$	
R_{100}	P_{100}	



thresholds

Thresholds

$[0.01, 0.02, \dots, 0.99, 1]$

th = 0.01 → Prec and Recall₁
 = 0.02 → Prec₂ and Recall₂
 = 0.03 → Prec₃ and Recall₃

ROC curve is not suitable for imbalanced datasets.

ROC curve TPR vs FPR

$$TPR = \frac{TP}{TP + FN}$$

$$FPR = \frac{FP}{FP + \textcircled{TN}}$$

class 0 $\rightarrow$ high population $\Rightarrow$ TN will be very very high
class 1 $\rightarrow$ low population

Since TN is very high $\Rightarrow$ FPR will always be low



Website: <https://inttrvu.ai>

Precision-recall curve

$$\text{Precision} = \frac{\text{TP}}{\text{TP} + \text{FP}}$$

$$\text{Recall} = \frac{\text{TP}}{\text{TP} + \text{FN}}$$

Using the custom thresholds

xgb-probabilities

	P0	P1	Prediction by xgboost (0.5)	custom (0.3)
R1	0.2	0.8	1	1
R2	0.4	0.5	1	1
R3	0.51	0.49	0	1
R4	0.62	0.38	0	1
R5	0.21	0.79	1	1
R6	0.1	0.9	1	1
R7	0.23	0.77	1	1
R8	0.25	0.75	1	0
R9	0.76	0.24	0	0
R10	0.99	0.01	0	0